# Sprint 2 - Work Summary

Made with GAMMA

# PANORAMICA DEI TASK SVILUPPATI

Nel progetto sono stati affrontati i seguenti task:

01

**Definire correttamente la Definition Of Done**

02

**Implementazione CRUD sui feedback**

03

**Integrazione e utilizzo di Codacy**

04

**Implementare Logout**

05

**Aggiungere informazioni su MFA nel claim del token**

Made with GAMMA

# TASK 1 – Definire correttamente Definition Of Done (#653)

## ANALISI

È stata rilevata la necessità di formalizzare una Definition of Done condivisa, al fine di:

- standardizzare i criteri di completamento dei task
- rendere oggettiva la valutazione delle Pull Request
- integrare qualità del codice, testing, documentazione e processo di review
- allineare il lavoro del team a pratiche di sviluppo strutturate

Made with GAMMA

# TASK 1 – Definire correttamente Definition Of Done (#653)

## IMPLEMENTAZIONE

# Definition of Done (DoD) - MS3

Criteri da soddisfare di cui ogni membro del team deve tener conto per poter considerare completata una User Story o un Task.

Made with GAMMA

# 1. Qualità del Codice (Code Quality)

- **Compilazione:** Il codice compila senza errori.
- **Analisi Statica (Codacy):**
  - Il **Quality Gate** definito su Codacy deve essere superato.
  - Il **Project Grade** non deve scendere sotto la soglia stabilita **D, quella già esistente**.
  - Il check automatico nella tab "Checks" della Pull Request su Github deve essere positivo.
  - Il codice deve rispettare le convenzioni standard verificate dai **Code Patterns** attivi nel progetto.

Made with GAMMA

# 2. Testing

### Unit Test

- Scritti per ogni nuova classe di logica.
- La copertura (Coverage) rispetta la soglia del **Quality Gate** configurato su Codacy.

### Integration Test

I test devono girare con successo con il database Docker (PostgreSQL).

### Test Cleanup

I test devono pulire i dati creati per non sporcare il DB.

### Test Manuale

Il test manuale della nuova funzionalità non deve presentare errori.

Made with GAMMA

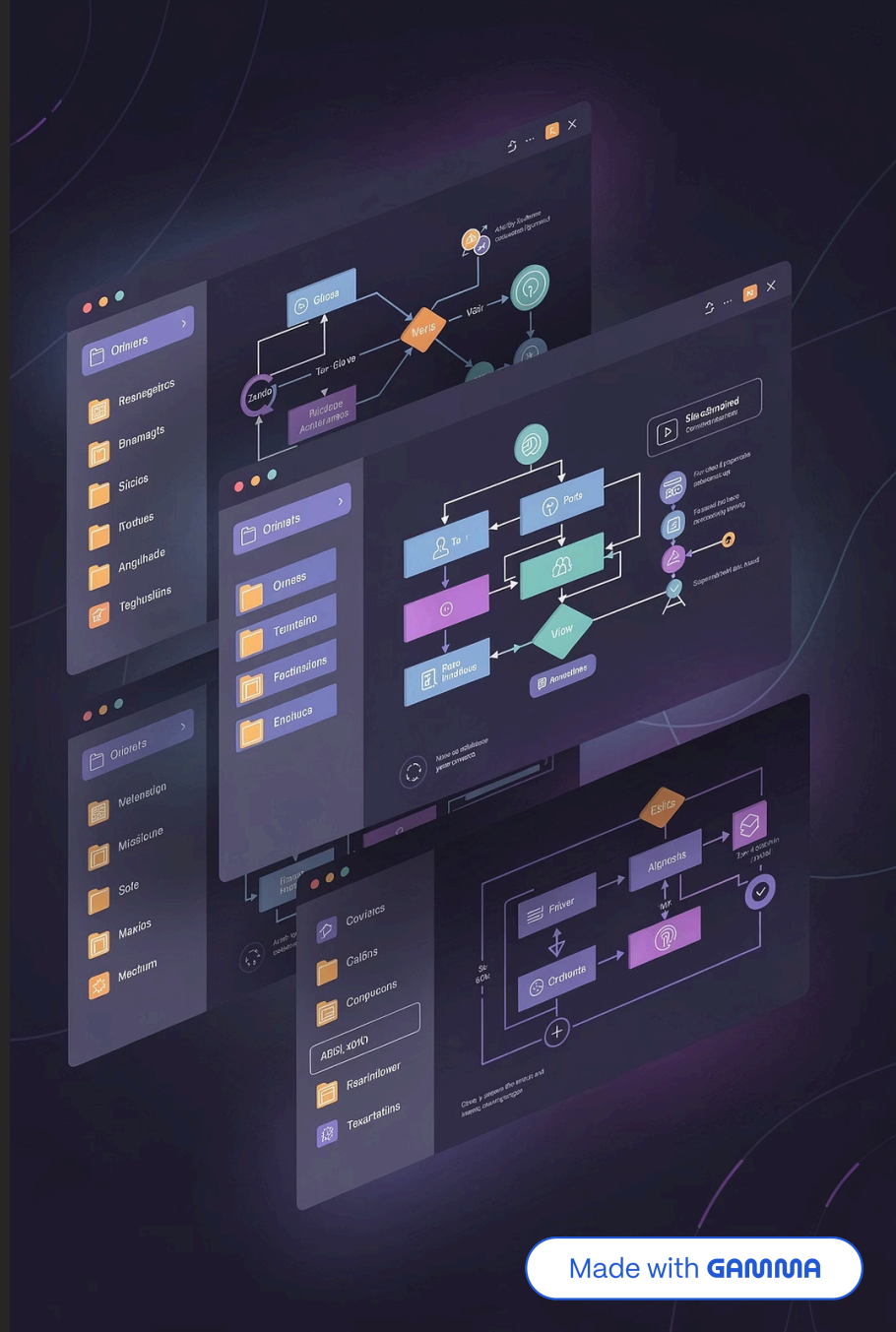
# 3. Documentazione

## Documentazione Tecnica

Aggiunta, aggiornamento del README o della documentazione tecnica.

## Diagrammi

Se la modifica impatta il modello dati, il diagramma ER su GitHub deve essere aggiornato.



Made with GAMMA

# 4. Processo & Review

- **Commit Message:** I messaggi devono essere chiari e parlanti (rispettare la convention sui commit message, e.g. numero dell'issue).
- **Deploy e Bug:** La funzionalità deve essere deployata senza errori e non devono esserci bug critici aperti relativi ad essa.
- **Pull Request:** La Pull Request verso il main branch deve essere approvata da almeno un altro membro del team.
- **Nessuna Regressione:** Tutti i test automatizzati esistenti devono passare con successo.

Eccezioni: Per task puramente documentali, si applicano solo le parti interessate delle sezioni 3 e 4.

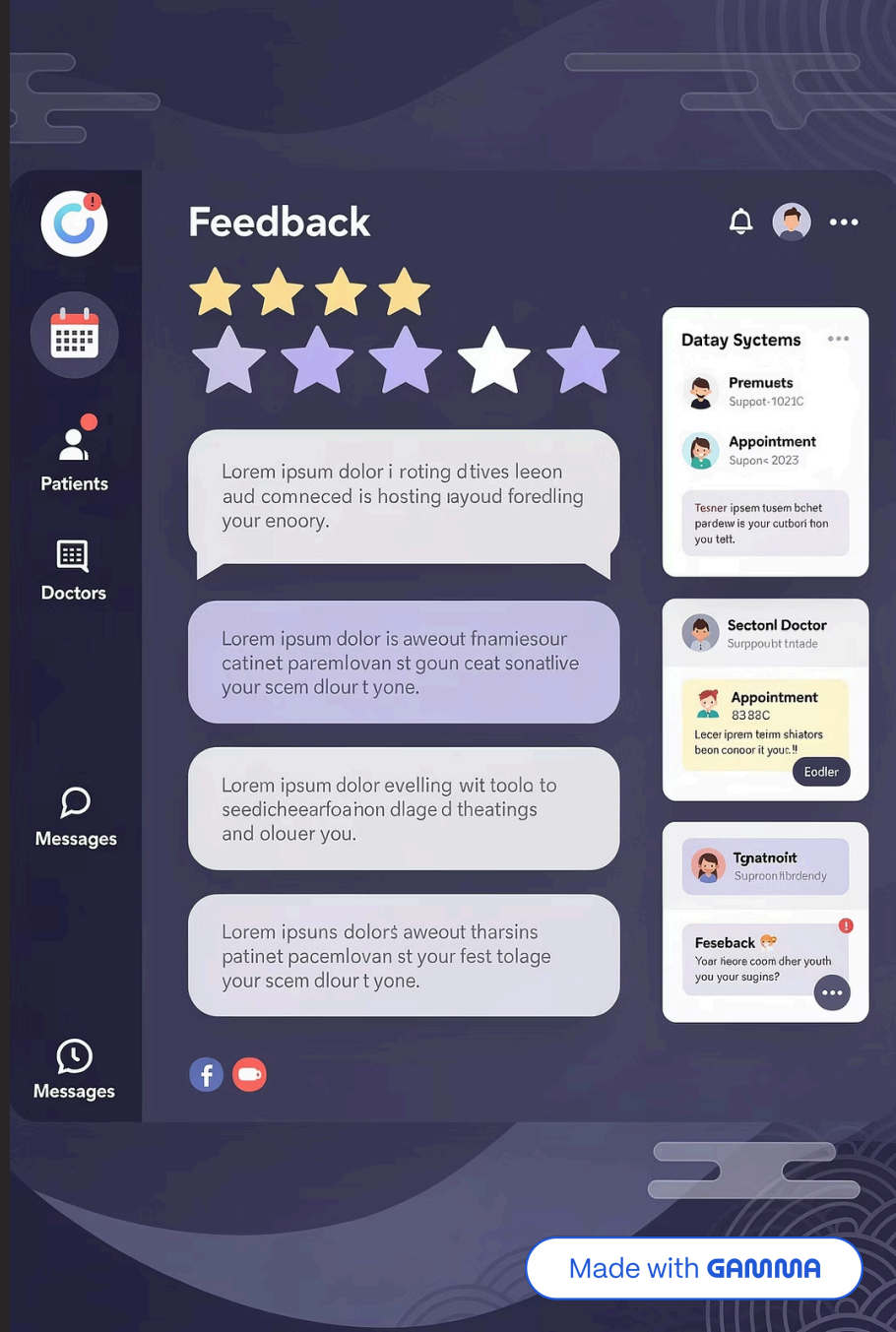
Made with GAMMA

# TASK 2 – Implementazione CRUD sui Feedback (#651)

## ANALISI

È stato progettato un sistema di feedback sui turni con i seguenti obiettivi:

- Raccolta strutturata delle valutazioni dei medici
- Separazione dei compiti Doctor / Planner
- Futura integrazione con agenti AI




Made with GAMMA

# TASK 2 – Implementazione CRUD sui Feedback (#651)

## IMPLEMENTAZIONE

### Backend

- Architettura rigorosa a strati (RestEndpoint → Controller → DAO → Entity)
- Entità ScheduleFeedback con relazioni ManyToMany verso i turni e ManyToOne verso il medico

### Ottimizzazione

- Identità utente estratta dal JWT
- Controllo accessi tramite RBAC e @PreAuthorize
- Query ottimizzate con strategie di Bulk Retrieval

### Frontend

- Inserimento feedback dal calendario del medico
- Dashboard amministrativa per il planner

### Testing

- Unit test ed integration test
- Verifica dei flussi principali e dei vincoli di sicurezza

Made with GAMMA

# TASK 2 – Implementazione CRUD sui Feedback (#651)

## COSA NON È STATO POSSIBILE IMPLEMENTARE

- Notifiche real-time per nuovi feedback
- Feedback strutturati per supporto a sistemi AI futuri

Made with GAMMA

# TASK 3 – Codacy (#656)

## ANALISI

La codebase presentava una forte presenza di codice legacy.

È stato necessario integrare Codacy senza bloccare lo sviluppo e chiarire il ruolo degli strumenti di qualità nel workflow.

Made with GAMMA

# TASK 3 – Codacy (#656)

## IMPLEMENTAZIONE

- Redazione di un manuale tecnico-operativo su Codacy
- Integrazione con GitHub Pull Request
- Allineamento con la Definition of Done
- Adozione dell'approccio New Issues Only
- Linee guida per il triage delle issue e la gestione del legacy

Made with GAMMA

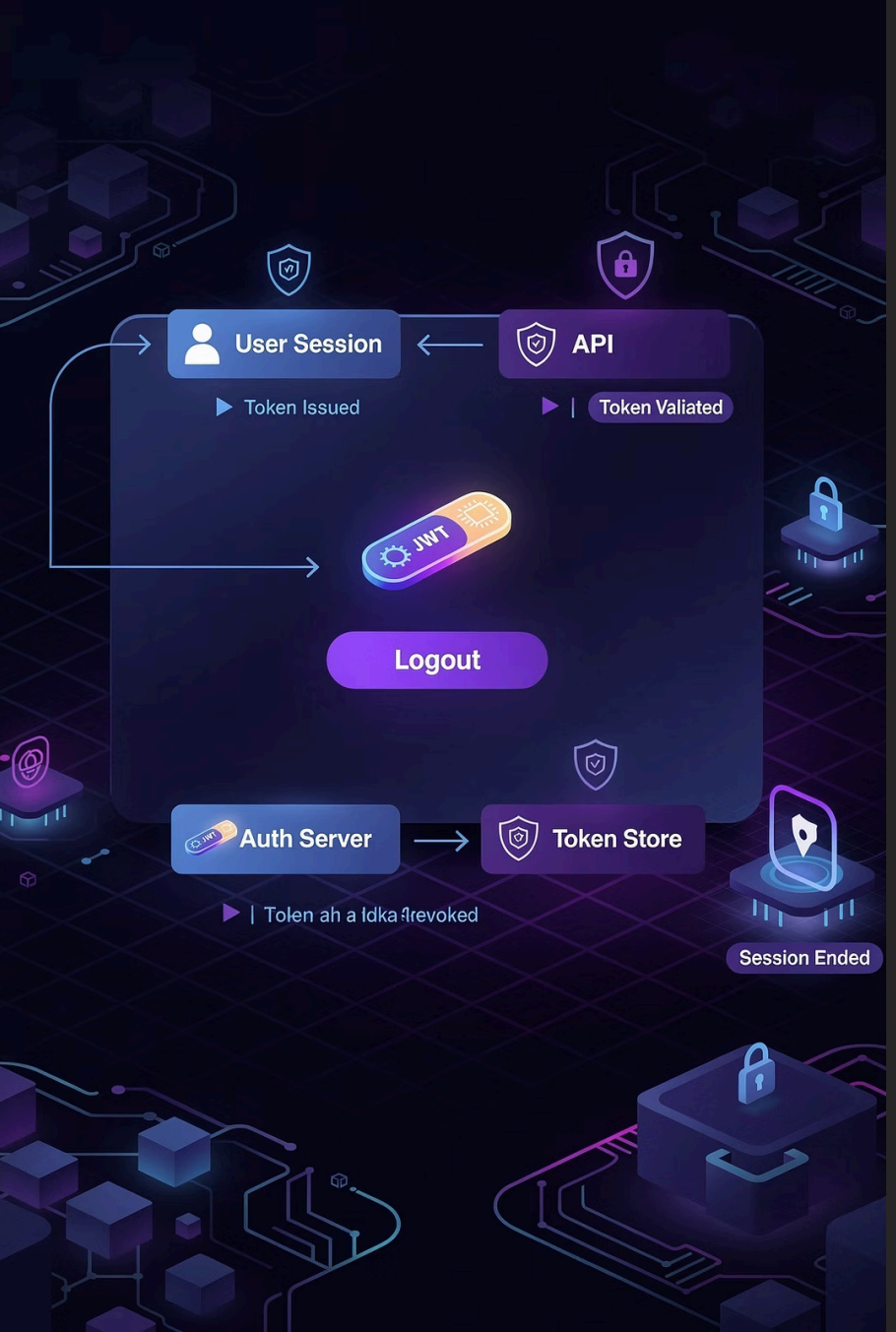
# TASK 3 – Codacy (#656)

## COSA NON È STATO POSSIBILE IMPLEMENTARE

Analisi approfondita dei Code Patterns:

- pattern attivi e inattivi
- scopo di ciascun pattern

Made with GAMMA




# TASK 4 – Implementare Logout (#621)

## ANALISI

È stato necessario introdurre una gestione avanzata dei token JWT per supportare il logout e l'invalidazione dei token utente.

Made with GAMMA

# TASK 4 – Implementare Logout (#621)

## IMPLEMENTAZIONE

1

### Estensione della tabella blacklisted_tokens

Associazione token–utente

2

### Token wildcard ("*")

Invalidazione globale dei token al cambio password

3

### Integrazione controlli

Validazione JWT e job schedulato per pulizia

- Integrazione dei controlli nella validazione JWT
- Job schedulato per la pulizia dei token scaduti
- Aggiornamento e creazione di test
- Documentazione sulla gestione dei JWT

Made with GAMMA

# TASK 5 – Aggiungere informazioni MFA nel claim del token (#654)

## ANALISI

I token di accesso non distinguevano tra autenticazione con MFA e senza MFA, limitando audit e controllo di sicurezza.

Made with GAMMA

# TASK 5 – Aggiungere informazioni MFA nel claim del token (#654)

## IMPLEMENTAZIONE

01

**Aggiunta dell'informazione MFA nei claim del token JWT**

02

**Integrazione nel flusso di autenticazione**

03

**Funzionalità completata con successo**

Made with GAMMA

# Grazie dell'attenzione

**CoffeeBreakpoint**

Made with GAMMA